

Proyecto 3 - VeriLang: Tipos y Sistema de Tipos

1. Fuente de la solución

Esta entrega parte directamente de nuestra solución del Proyecto 2. No se utilizó la solución de ningún otro grupo. Los archivos Syntax.rsc y AST.rsc del Proyecto 2 fueron la base para todas las extensiones descritas en este documento.

2. Correcciones con respecto al Proyecto 2

2.1 Resaltamiento de sintaxis (highlighting)

El comentario del calificador del Proyecto 2 fue: "No es posible probar el resaltamiento de gramática. En futuras ocasiones, por favor incluyan los archivos del proyecto completo."

El problema fue que en la entrega anterior no se incluyeron las anotaciones de resaltamiento en el archivo Syntax.rsc, lo que hizo imposible verificar el highlighting desde el IDE. Para el Proyecto 3 se agregaron las siguientes anotaciones al final del archivo Syntax.rsc:

```
keyword highlight Keywords      = \category("keyword");
keyword highlight BoolLit       = \category("constant");
lexical  highlight IntLit       = \category("constant.numeric");
lexical  highlight CharLit      = \category("string.quoted.single");
lexical  highlight StringLit    = \category("string.quoted.double");
keyword  highlight BuiltinType = \category("storage.type");
```

Con estas anotaciones, al abrir el proyecto en VS Code con la extensión de Rascal, cada categoría de token recibe el color correspondiente: palabras reservadas en un color, literales numéricos en otro, strings en otro, y los tipos (Int, Bool, Char, String) en otro distinto.

Adicionalmente, para esta entrega se incluye el proyecto Rascal completo (todos los archivos .rsc, el META-INF/RASCAL.MF y los ejemplos .vl) para que el calificador pueda abrir el proyecto directamente y verificar el resaltamiento.

2.2 Cuantificadores no restringidos a expresiones de contención

La rubrica del Proyecto 2 señaló: "Los cuantificadores no deberían de estar limitados a solo expresiones de contención."

En la gramática original del Proyecto 2, forall y exists solo podían aparecer seguidos de una expresión de membresía ($x \in S$). Esto era demasiado restrictivo para un lenguaje de especificación matemática. Se modificó la regla de Expr para que el cuerpo del cuantificador pueda ser cualquier expresión:

```

syntax Expr
  = quantifiedExpr: Quantifier Id "in" Id "." Expr
  | infixExpr:      InfixExpr
  ;

```

De esta forma, expresiones como "forall x in Naturals . (plus x 0) = x" son validas, sin importar la forma de la expresion en el cuerpo.

3. Cambios implementados para el Proyecto 3

3.1 Literales con anotacion de tipo en la gramatica

Se agregaron cuatro nuevos no terminales lexicos para representar valores literales concretos en el texto del programa:

- IntLit: uno o mas digitos decimales. Ejemplo: 42
- BoolLit: las palabras true o false
- CharLit: un caracter entre comillas simples. Ejemplo: 'a'
- StringLit: cero o mas caracteres entre comillas dobles. Ejemplo: "hola"

En SimpleExpr se agregaron cuatro alternativas nuevas para que un literal lleve su tipo anotado explicitamente en el programa:

```

| simpleInt:      IntLit      ":" "Int"
| simpleBool:     BoolLit     ":" "Bool"
| simpleChar:     CharLit     ":" "Char"
| simpleString:   StringLit   ":" "String"

```

Ejemplo de uso en un programa .vl:

```

defexpression x = 42:Int end
defexpression flag = true:Bool end
defexpression inicial = 'a':Char end
defexpression saludo = "hola":String end

```

3.2 Tipo BuiltinType en la gramatica y el AST

Se creo el no terminal BuiltinType para representar los cuatro tipos primitivos requeridos (Int, Bool, Char, String) y tambien tipos definidos por el usuario (identificadores). Este no terminal se usa en las declaraciones de variables y en las estructuras de datos:

```

syntax BuiltinType
  = typeInt:      "Int"

```

```

| typeBool:    "Bool"
| typeChar:    "Char"
| typeString:  "String"
| typeId:      Id
;

```

En el AST se agrego el tipo de dato correspondiente:

```

data BuiltinType
= typeInt()
| typeBool()
| typeChar()
| typeString()
| typeId(str name)
;

```

3.3 Anotaciones de tipo en variables (VarBinding)

La regla VarBinding se actualizo para que el tipo de la variable sea un BuiltinType en lugar de un identificador generico. Esto permite al parser y al type checker validar que el tipo declarado sea uno de los tipos reconocidos:

```

// Antes (Proyecto 2):
syntax VarBinding = varBinding: Id ":" Id;

// Ahora (Proyecto 3):
syntax VarBinding = varBinding: Id ":" BuiltinType;

```

Ejemplo:

```

defvar
  x      : Int,
  flag   : Bool,
  letra  : Char,
  nombre : String
end

```

3.4 Estructuras de datos con campos tipados (DataDecl)

Se agrego un nuevo tipo de componente al lenguaje: la declaracion de estructura de datos (defdata). Esta permite al usuario definir una estructura con nombre, un tipo de usuario, y una lista de campos tipados:

```

syntax DataDecl
= dataDecl: "defdata" Id ":" BuiltinType DataFieldList "end"

```

```

;

syntax DataField = dataField: Id ":" BuiltinType;

```

Ejemplo de uso:

```

defdata Punto : Punto
  px : Int
  py : Int
end

defdata Persona : Persona
  edad : Int
  nombre : String
end

```

3.5 Parser (Parser.rsc)

Se implemento el modulo Parser.rsc que utiliza la funcion parse de Rascal para convertir el texto fuente en un arbol de derivacion concreto, y luego implode para transformarlo en el AST. Se ofrecen dos funciones de entrada:

- parseProgram(str src): parsea un programa desde un String
- parseProgramFile(loc file): parsea un programa desde un archivo .vl

El parser se puede usar directamente desde el REPL de Rascal para verificar que un programa es valido:

```

import verilang::Parser;
parseProgram("defmodule Test defvar x : Int end end");

```

3.6 Type checker con TypePal (Checker.rsc)

Se instalo TypePal en el lenguaje mediante el modulo Checker.rsc. El checker implementa tres funcionalidades principales:

a) Representacion interna de tipos:

Se definio el tipo AType con los cuatro tipos primitivos y soporte para tipos de usuario. Se implemento una funcion de subtipos (mySubtype) que exige igualdad exacta entre tipos, y una funcion de impresion (prettyAType) para mensajes de error legibles.

b) Verificacion de correspondencia de tipos:

En la fase de recoleccion (collect), cada declaracion de variable registra su nombre y tipo con TypePal usando define(). Cada uso de una variable en una expresion se registra con use(). Cada literal anotado registra su tipo con fact(). TypePal resuelve automaticamente si el tipo usado corresponde al tipo declarado y reporta errores cuando no coinciden.

c) Regla de existencia de elementos en estructuras de datos:

Se implemento la funcion checkDataFieldExistence que verifica que todos los campos referenciados en expresiones cuantificadas sobre una estructura de datos existan realmente en la declaracion de esa estructura. Por ejemplo, si se escribe:

```
defdata Punto : Punto  px : Int  py : Int  end
defexpression forall p in Punto . p.pz = 0 end
```

El checker reportaria un error porque pz no es un campo declarado en Punto.

3.7 Generacion de codigo y ejecucion (CodeGen.rsc y Main.rsc)

Se implemento CodeGen.rsc que recorre el AST e imprime en consola el contenido del programa de forma estructurada. Para cada componente del modulo se muestra su tipo, nombre y valor. El punto de entrada es Main.rsc:

```
import verilang::Main;
main(|project://verilang-project3/examples/example.vl|);
```

La salida en consola tiene el siguiente formato:

```
=== VeriLang program: MyModule ===
[Space]  Naturals
[Space]  Integers  <  Naturals
[Operator]  plus : Naturals -> Naturals -> Naturals
[Var]  x : Int
[Var]  flag : Bool
[Data]  Punto : Punto
        field  px : Int
        field  py : Int
[Expression]  x = 42:Int
[Equation]  (plus x y)  =  (plus y x)
=== End of program ===
```

4. Resumen de archivos entregados

Archivo	Descripcion
---------	-------------

src/verilang/Syntax.rsc	Gramatica actualizada con tipos, literales anotados, defdata y highlighting
src/verilang/AST.rsc	Sintaxis abstracta actualizada (BuiltinType, DataDecl, SimpleExpr con literales)
src/verilang/Parser.rsc	Parser: convierte texto .vl en AST mediante parse + implode
src/verilang/Checker.rsc	Type checker con TypePal: tipos, correspondencia y existencia de campos
src/verilang/CodeGen.rsc	Generador de salida: imprime el programa en consola
src/verilang/Main.rsc	Punto de entrada principal para correr archivos .vl
META-INF/RASCAL.MF	Archivo de configuracion del proyecto Rascal
examples/example.vl	Programa de ejemplo que ejercita todas las funcionalidades nuevas